데이터 베이스 시스템
Project2

교수님 : 정성원 교수님
이름 : 김민우
학번 : 20212019
전공 : 컴퓨터공학과

1. 프로젝트 개요

부동산 거래 시스템의 데이터베이스 설계를 목표로 한다.

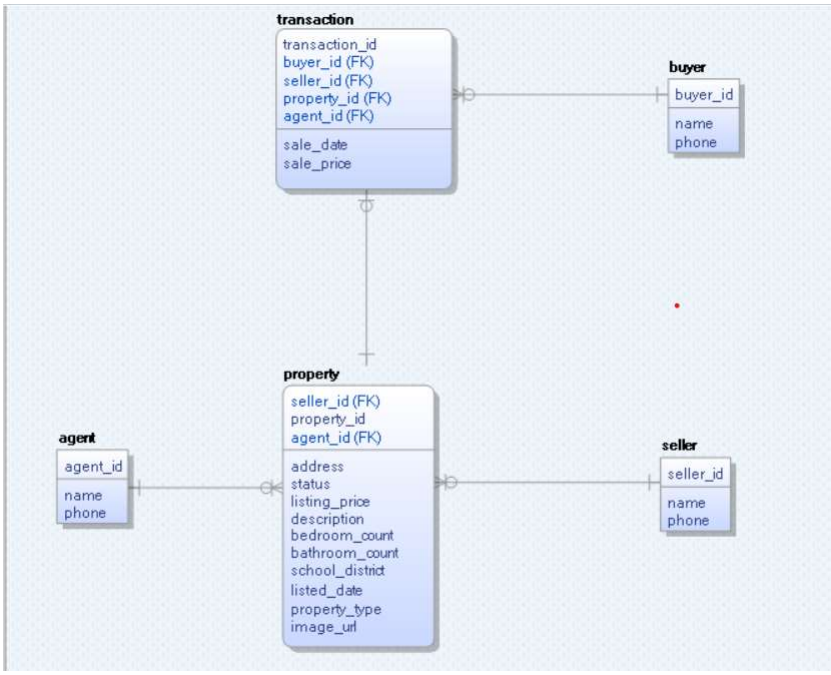
프로젝트는 크게 두 부분으로 나뉘며, 첫 번째 부분에서는 ER 다이어그램을 작성하여 데이터 모델을 개념적으로 설계하고, 두 번째 부분에서는 논리적 및 물리적 스키마로 변환하여 구현 가능한 형태로 만드는 과정을 거치게 된다.

프로젝트 1에서는 다음과 같은 작업이 수행되었다 :

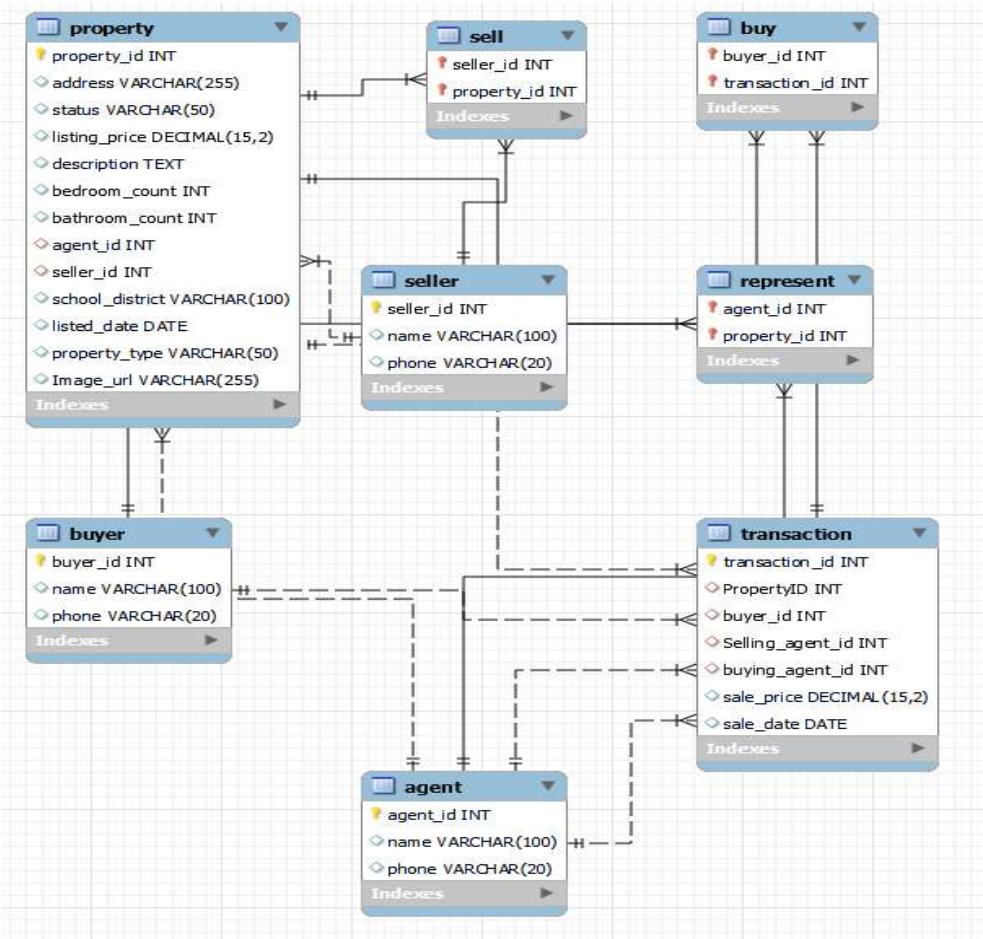
부동산 거래 시스템의 주요 엔터티와 관계를 식별하고 ER 다이어그램을 작성하며 각 엔터티의 속성 및 관계를 정의.

프로젝트 2에서는 이를 기반으로 논리적 다이어그램을 설계하고 BCNF(Boyce-Codd Normal Form) 분해를 통해 데이터 무결성을 높였다. 논리적 스키마를 물리적 스키마로 변환하여 실제 데이터베이스 시스템에 적용하고, ODBC를 사용하여 MySQL 데이터베이스에 연결하고 CRUD(생성, 조회, 갱신, 삭제) 쿼리를 수행하였다.

[project 1 schema]



[project 2 Decomposed Logical Schema]



2. 프로젝트 1에서 프로젝트 2로 변환 및 BCNF 분해 설명

* 프로젝트 1의 Schema

프로젝트 1에서는 부동산 거래 시스템의 주요 엔터티와 관계를 식별하고 ER 다이어그램을 작성하였다. 주요 엔터티와 관계는 다음과 같다:

Transaction: 거래 정보를 저장 (transaction_id, buyer_id, seller_id, property_id, agent_id, sale_date, sale_price)
Property: 부동산 정보를 저장 (property_id, seller_id, agent_id, address, status, listing_price, description, bedroom_count, bathroom_count, school_district, listed_date, property_type, image_url)
Agent: 부동산 중개인 정보를 저장 (agent_id, name, phone)
Buyer: 구매자 정보를 저장 (buyer_id, name, phone)
Seller: 판매자 정보를 저장 (seller_id, name, phone)
이 스키마는 시스템의 주요 개체 및 그들 간의 관계를 표현하였다.

* 프로젝트 2의 Logical Schema

프로젝트 2에서는 기존 스키마를 더 구체화하고, 복잡한 엔터티를 분해하여 관리 가능한 수준으로 만들었다. 주요 추가점과 변경 사항은 다음과 같다:

Transaction 엔터티에서 selling_agent_id와 buying_agent_id를 추가하여 판매 중개인과 구매 중개인을 명확히 구분하였습니다.
새로운 중간 엔터티를 추가하여 관계를 명확히 하였습니다:
Buy: 구매자와 거래 간의 관계
Sell: 판매자와 거래 간의 관계
Represent: 부동산 중개인과 부동산 간의 관계
Property Transaction: 부동산과 거래 간의 관계

* 각 Entity의 BCNF 분해

각 Entity에 대해 BCNF(Boyce-Codd Normal Form) 분해를 진행하여 데이터 무결성을 유지하였다. 각 Entity의 함수적 종속성을 분석하고 BCNF를 위반하는 경우 분해를 진행하였다.

① Transaction Entity

기존 속성: transaction_id, property_id, buyer_id, seller_id, agent_id, sale_date, sale_price

추가 속성: selling_agent_id, buying_agent_id

함수적 종속성: transaction_id -> property_id, buyer_id, seller_id, selling_agent_id, buying_agent_id, sale_date, sale_price

이 함수적 종속성은 BCNF를 위반하지 않으므로 분해가 필요하지 않다.

② Property Entity

기존 속성: property_id, seller_id, agent_id, address, status, listing_price, description, bedroom_count, bathroom_count, school_district, listed_date, property_type, image_url

함수적 종속성: property_id -> seller_id, agent_id, address, status, listing_price, description, bedroom_count, bathroom_count, school_district, listed_date, property_type, image_url

이 함수적 종속성은 BCNF를 위반하지 않으므로 분해가 필요하지 않다.

③ Agent Entity

기존 속성: agent_id, name, phone

함수적 종속성: agent_id -> name, phone

이 함수적 종속성은 BCNF를 위반하지 않으므로 분해가 필요하지 않다.

④ Buyer Entity

기존 속성: buyer_id, name, phone

함수적 종속성: buyer_id -> name, phone

이 함수적 종속성은 BCNF를 위반하지 않으므로 분해가 필요하지 않다.

⑤ Seller Entity

기존 속성: seller_id, name, phone

함수적 종속성:

seller_id -> name, phone

이 함수적 종속성은 BCNF를 위반하지 않으므로 분해가 필요하지 않다.

* 추가된 Entity

프로젝트 2의 Logical Schema에서 추가된 Entity는 다음과 같다:

Buy: transaction_id, buyer_id

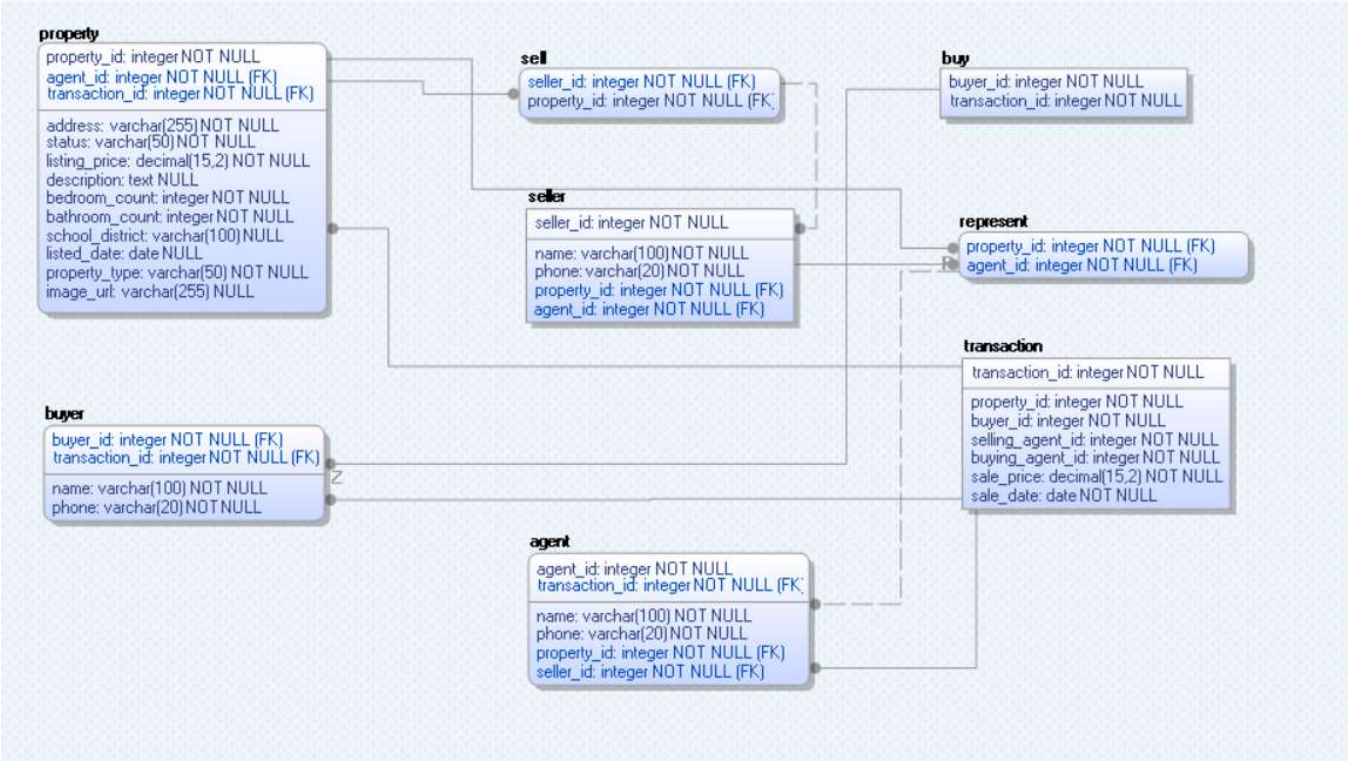
Sell: transaction_id, seller_id

Represent: agent_id, property_id

Property Transaction: transaction_id, property_id

이러한 중간 엔터티를 추가함으로써 각 관계를 더 명확하게 정의하고 관리할 수 있게 되었다.

3. Physical Schema



physical schema는 logical schema를 바탕으로 설계 되었고, 각 table과 relation에 data type, relation type, null 여부등을 기재하였다. 다음은 그에대한 상세한 설명이다.

① property : 부동산 관련 정보를 저장하는 테이블

- property_id (integer, NOT NULL, Primary Key)

각 부동산의 고유 식별자이다. 고유 식별자는 정수형이 효율적이고 빠른 검색을 가능하게 한다. 식별자는 항상 존재해야 하므로 NULL 값을 허용하지 않는다.

- address (varchar(255), NOT NULL)

부동산의 주소이다. 주소는 가변 길이의 문자열이므로 varchar 타입을 사용한다. 주소는 필수 정보이므로 NULL 값을 허용하지 않는다.

- status (varchar(50), NOT NULL)

부동산의 현재 상태이다 (예: 'available', 'sold'). 상태 값이 비교적 짧은 텍스트이므로

varchar(50)으로 지정한다. 상태는 반드시 지정되어야 하므로 NULL 값을 허용하지 않는다.

- listing_price (decimal(15,2), NOT NULL)

부동산의 판매 가격이다. 금액을 저장할 때는 소수점 이하 두 자리까지 정확하게 표현할 수 있는 decimal 타입을 사용한다. 판매 가격은 필수 정보이므로 NULL 값을 허용하지 않는다.

- description (text, NULL)

부동산에 대한 상세 설명이다. 설명은 길이가 가변적이고 길 수 있으므로 text 타입을 사용한다. 설명은 선택 사항이므로 NULL 값을 허용한다.

- bedroom_count (integer, NOT NULL)

부동산의 침실 수이다. 침실 수는 정수형 데이터이므로 integer 타입을 사용한다. 침실 수는 필수 정보이므로 NULL 값을 허용하지 않는다.

- bathroom_count (integer, NOT NULL)

부동산의 욕실 수이다. 욕실 수 역시 정수형 데이터이므로 integer 타입을 사용한다. 욕실 수는 필수 정보이므로 NULL 값을 허용하지 않는다.

- agent_id (integer, NOT NULL, Foreign Key)

부동산을 담당하는 에이전트의 ID이다. 외래 키로 다른 테이블의 정수형 ID를 참조하므로 integer 타입을 사용한다. 담당 에이전트는 필수 정보이므로 NULL 값을 허용하지 않는다.

- school_district (varchar(100), NULL)

부동산이 위치한 학군이다. 학군 이름은 가변 길이의 문자열이므로 varchar(100) 타입을 사용한다. 학군 정보는 선택 사항이므로 NULL 값을 허용한다.

- listed_date (date, NULL)

부동산이 시장에 나온 날짜이다. 날짜 데이터이므로 date 타입을 사용한다. 리스트 날짜는 선

택 사항이므로 NULL 값을 허용한다.

-property_type (varchar(50), NOT NULL)
부동산의 유형이다 (예: 'apartment', 'house'). 유형은 짧은 텍스트이므로 varchar(50) 타입을 사용한다. 부동산 유형은 필수 정보이므로 NULL 값을 허용하지 않는다.

-image_url (varchar(255), NULL): 부동산 이미지를 저장하는 URL이다. URL은 가변 길이의 문자열이므로 varchar(255) 타입을 사용한다. 이미지 URL은 선택 사항이므로 NULL 값을 허용한다.

② seller : 부동산 판매자의 정보를 저장하는 테이블

- seller_id (integer, NOT NULL, Primary Key)
판매자의 고유 식별자이다. 고유 식별자는 정수형이 효율적이고 빠른 검색을 가능하게 한다. 식별자는 항상 존재해야 하므로 NULL 값을 허용하지 않는다.

-name (varchar(100), NOT NULL)
판매자의 이름이다. 이름은 가변 길이의 문자열이므로 varchar(100) 타입을 사용한다. 이름은 필수 정보이므로 NULL 값을 허용하지 않는다.

-phone (varchar(20), NOT NULL)
판매자의 전화번호이다. 전화번호는 가변 길이의 문자열이므로 varchar(20) 타입을 사용한다. 전화번호는 필수 정보이므로 NULL 값을 허용하지 않는다.

③ sell : 부동산 판매 정보를 저장하는 테이블

- seller_id (integer, NOT NULL, Foreign Key)
판매자의 ID이다. 외래 키로 다른 테이블의 정수형 ID를 참조하므로 integer 타입을 사용한다. 판매자 ID는 필수 정보이므로 NULL 값을 허용하지 않는다.

-property_id (integer, NOT NULL, Foreign Key)

판매된 부동산의 ID이다. 외래 키로 다른 테이블의 정수형 ID를 참조하므로 integer 타입을 사용한다. 부동산 ID는 필수 정보이므로 NULL 값을 허용하지 않는다.

④ buyer : 부동산 구매자의 정보를 저장하는 테이블

- buyer_id (integer, NOT NULL, Primary Key)

구매자의 고유 식별자이다. 고유 식별자는 정수형이 효율적이고 빠른 검색을 가능하게 한다. 식별자는 항상 존재해야 하므로 NULL 값을 허용하지 않는다.

- name (varchar(100), NOT NULL)

구매자의 이름이다. 이름은 가변 길이의 문자열이므로 varchar(100) 타입을 사용한다. 이름은 필수 정보이므로 NULL 값을 허용하지 않는다.

- phone (varchar(20), NOT NULL)

구매자의 전화번호이다. 전화번호는 가변 길이의 문자열이므로 varchar(20) 타입을 사용한다. 전화번호는 필수 정보이므로 NULL 값을 허용하지 않는다.

⑤ buy : 부동산 구매 정보를 저장하는 테이블

- buyer_id (integer, NOT NULL, Foreign Key)

구매자의 ID이다. 외래 키로 다른 테이블의 정수형 ID를 참조하므로 integer 타입을 사용한다. 구매자 ID는 필수 정보이므로 NULL 값을 허용하지 않는다.

- transaction_id (integer, NOT NULL, Foreign Key)

구매와 관련된 거래의 ID이다. 외래 키로 다른 테이블의 정수형 ID를 참조하므로 integer 타입을 사용한다. 거래 ID는 필수 정보이므로 NULL 값을 허용하지 않는다.

⑥ agent : 부동산 에이전트의 정보를 저장하는 테이블

- agent_id (integer, NOT NULL, Primary Key)

에이전트의 고유 식별자이다. 고유 식별자는 정수형이 효율적이고 빠른 검색을 가능하게 한다. 식별자는 항상 존재해야 하므로 NULL 값을 허용하지 않는다.

- name (varchar(100), NOT NULL)

에이전트의 이름이다. 이름은 가변 길이의 문자열이므로 varchar(100) 타입을 사용한다. 이름은 필수 정보이므로 NULL 값을 허용하지 않는다.

- phone (varchar(20), NOT NULL)

에이전트의 전화번호이다. 전화번호는 가변 길이의 문자열이므로 varchar(20) 타입을 사용한다. 전화번호는 필수 정보이므로 NULL 값을 허용하지 않는다.

⑦ represent : 에이전트와 부동산 간의 관계를 저장하는 테이블

- agent_id (integer, NOT NULL, Foreign Key)

에이전트의 ID이다. 외래 키로 다른 테이블의 정수형 ID를 참조하므로 integer 타입을 사용한다. 에이전트 ID는 필수 정보이므로 NULL 값을 허용하지 않는다.

- property_id (integer, NOT NULL, Foreign Key)

에이전트가 대표하는 부동산의 ID이다. 외래 키로 다른 테이블의 정수형 ID를 참조하므로 integer 타입을 사용한다. 부동산 ID는 필수 정보이므로 NULL 값을 허용하지 않는다.

⑧ transaction : 부동산 거래 정보를 저장하는 테이블

- transaction_id (integer, NOT NULL, Primary Key)

거래의 고유 식별자이다. 고유 식별자는 정수형이 효율적이고 빠른 검색을 가능하게 한다. 식별자는 항상 존재해야 하므로 NULL 값을 허용하지 않는다.

- property_id (integer, NOT NULL, Foreign Key)

거래된 부동산의 ID이다. 외래 키로 다른 테이블의 정수형 ID를 참조하므로 integer 타입을 사용한다. 부동산 ID는 필수 정보이므로 NULL 값을 허용하지 않는다.

- buyer_id (integer, NOT NULL, Foreign Key)

거래에 참여한 구매자의 ID이다. 외래 키로 다른 테이블의 정수형 ID를 참조하므로 integer 타입을 사용한다. 구매자 ID는 필수 정보이므로 NULL 값을 허용하지 않는다.

- selling_agent_id (integer, NOT NULL, Foreign Key)

판매를 담당한 에이전트의 ID이다. 외래 키로 다른 테이블의 정수형 ID를 참조하므로 integer 타입을 사용한다. 판매 에이전트 ID는 필수 정보이므로 NULL 값을 허용하지 않는다.

- buying_agent_id (integer, NOT NULL, Foreign Key)

구매를 담당한 에이전트의 ID이다. 외래 키로 다른 테이블의 정수형 ID를 참조하므로 integer 타입을 사용한다. 구매 에이전트 ID는 필수 정보이므로 NULL 값을 허용하지 않는다.

- sale_price (decimal(15,2), NOT NULL)

거래된 부동산의 판매 가격이다. 금액을 저장할 때는 소수점 이하 두 자리까지 정확하게 표현할 수 있는 decimal 타입을 사용한다. 판매 가격은 필수 정보이므로 NULL 값을 허용하지 않는다.

- sale_date (date, NOT NULL)

거래가 이루어진 날짜이다. 날짜 데이터이므로 date 타입을 사용한다. 거래 날짜는 필수 정보이므로 NULL 값을 허용하지 않는다.

4. ODBC implement within MySQL

① crud를 위한 text file

-20212019_1.txt

20212019_1.txt 20212019_2.txt

파일 편집 보기

```
CREATE TABLE Seller (seller_id INT PRIMARY KEY, name VARCHAR(100), phone
VARCHAR(20));
CREATE TABLE Property (property_id INT PRIMARY KEY, address VARCHAR(255), status
VARCHAR(50), listing_price DECIMAL(15, 2), description TEXT, bedroom_count INT,
bathroom_count INT, agent_id INT, seller_id INT, school_district VARCHAR(100), listed_date
DATE, property_type VARCHAR(50), Image_url VARCHAR(255), FOREIGN KEY (agent_id)
REFERENCES Agent(agent_id), FOREIGN KEY (seller_id) REFERENCES Seller(seller_id));
CREATE TABLE Transaction (transaction_id INT PRIMARY KEY, PropertyID INT, buyer_id INT,
Selling_agent_id INT, buying_agent_id INT, sale_price DECIMAL(15, 2), sale_date DATE,
FOREIGN KEY (PropertyID) REFERENCES Property(property_id), FOREIGN KEY (buyer_id)
REFERENCES Buyer(buyer_id), FOREIGN KEY (Selling_agent_id) REFERENCES Agent(agent_id),
FOREIGN KEY (buying_agent_id) REFERENCES Agent(agent_id));
CREATE TABLE Represent (agent_id INT, property_id INT, PRIMARY KEY (agent_id,
property_id), FOREIGN KEY (agent_id) REFERENCES Agent(agent_id), FOREIGN KEY
(property_id) REFERENCES Property(property_id));
CREATE TABLE Sell (seller_id INT, property_id INT, PRIMARY KEY (seller_id, property_id),
FOREIGN KEY (seller_id) REFERENCES Seller(seller_id), FOREIGN KEY (property_id)
REFERENCES Property(property_id));
CREATE TABLE Buy (buyer_id INT, transaction_id INT, PRIMARY KEY (buyer_id,
transaction_id), FOREIGN KEY (buyer_id) REFERENCES Buyer(buyer_id), FOREIGN KEY
(transaction_id) REFERENCES Transaction(transaction_id));
INSERT INTO Property (property_id, address, status, listing_price, description,
bedroom_count, bathroom_count, agent_id, seller_id, school_district, listed_date,
property_type, Image_url) VALUES (1, '123 Main St, Seoul', 'For Sale', 500000000, 'Cozy
apartment in the city center', 2, 1, 1, 1, '5', '2022-01-01', 'one_bedroom', 'image1.jpg');
INSERT INTO Property (property_id, address, status, listing_price, description,
bedroom_count, bathroom_count, agent_id, seller_id, school_district, listed_date,
```

20212019_1.txt 20212019_2.txt

파일 편집 보기

```
ALTER TABLE Property DROP FOREIGN KEY Property_ibfk_1;
ALTER TABLE Property DROP FOREIGN KEY Property_ibfk_2;
ALTER TABLE Transaction DROP FOREIGN KEY Transaction_ibfk_1;
ALTER TABLE Transaction DROP FOREIGN KEY Transaction_ibfk_2;
ALTER TABLE Transaction DROP FOREIGN KEY Transaction_ibfk_3;
ALTER TABLE Transaction DROP FOREIGN KEY Transaction_ibfk_4;
ALTER TABLE Represent DROP FOREIGN KEY Represent_ibfk_1;
ALTER TABLE Represent DROP FOREIGN KEY Represent_ibfk_2;
ALTER TABLE Sell DROP FOREIGN KEY Sell_ibfk_1;
ALTER TABLE Sell DROP FOREIGN KEY Sell_ibfk_2;
ALTER TABLE Buy DROP FOREIGN KEY Buy_ibfk_1;
ALTER TABLE Buy DROP FOREIGN KEY Buy_ibfk_2;
DROP TABLE IF EXISTS Property;
DROP TABLE IF EXISTS Transaction;
DROP TABLE IF EXISTS Agent;
DROP TABLE IF EXISTS Buyer;
DROP TABLE IF EXISTS Seller;
DROP TABLE IF EXISTS Represent;
DROP TABLE IF EXISTS Sell;
DROP TABLE IF EXISTS Buy;
```

파일 20212019_1.txt는 테이블 생성과 데이터 삽입을 위한 SQL 쿼리들을 포함하고 있다. 이 파일에는 Agent, Buyer, Seller, Property, Transaction, Represent, Sell, Buy 테이블을 생성하는 쿼리들이 포함되어 있으며, 각 테이블은 고유 식별자(Primary Key)를 가지며 외래 키(Foreign Key) 관계가 설정되어 있습. 또한, 이 파일에는 각 테이블에 초기 데이터를 삽입하는 SQL 쿼리들이 포함되어 있어, 데이터베이스 구조를 정의하고 초기 데이터를 삽입하여 프로젝트에서 필요한 데이터를 미리 준비하는 역할을 한다. 예를 들어, Property 테이블에 여러 부동산 정보를 삽입하는 쿼리와 Agent 테이블에 여러 에이전트 정보를 삽입하는 쿼리 등이 포함되어 있다.

파일 20212019_2.txt는 외래 키 제약 조건을 제거하고 테이블을 삭제하는 SQL 쿼리들을 포함하고 있다. 이 파일에서는 각 테이블에서 외래 키 제약 조건을 제거한 후, 테이블을 삭제한다. Property, Transaction, Agent, Buyer, Seller, Represent, Sell, Buy 테이블을 순차적으로 삭제하는 쿼리들이 포함되어 있다. 외래 키 제약 조건을 제거하는 이유는 테이블을 안전하게 삭제하기 위함이다. 외래 키가 설정된 상태에서 테이블을 삭제하려고 하면 참조 무결성 오류가 발생할 수 있기 때문에, 이를 방지하기 위해 먼저 외래 키 제약 조건을 제거하는 것이다. 이 두 파일을 통해 데이터베이스의 테이블을 생성하고 초기 데이터를 삽입한 후, 필요시 테이블과 외래 키 제약 조건을 제거하여 데이터베이스를 초기 상태로 되돌릴 수 있다.

② 각 query에 대한 결과 값

TYPE 1: Find address of homes for sale in the district “Mapo”

sql :

```
SELECT address FROM Property WHERE address LIKE '%Mapo%';
```

결과 :

```
123 Mapo St, Seoul
456 Mapo Ave, Seoul
```

TYPE 1-1: Then find the costing between ₩1,000,000,000 and ₩1,500,000,000

sql :

```
SELECT address FROM Property WHERE address LIKE '%Mapo%' AND listing_price
BETWEEN 1000000000 AND 1500000000;
```

결과 :

```
123 Mapo St, Seoul
```

TYPE 2: Find the address of homes for sale in the 8th school district

sql :

```
SELECT address FROM Property WHERE school_district = '8';
```

결과 :

```
789 Pine Rd, Daejeon
```

TYPE 2-1: Then find properties with 4 or more bedrooms and 2 bathrooms

sql :

```
SELECT address FROM Property WHERE school_district = '8' AND bedroom_count >= 4
AND bathroom_count = 2;
```

결과 :

```
789 Pine Rd, Daejeon
```

TYPE 3: Find the name of the agent who has sold the most properties in the year 2022 by total won value

sql :

```
SELECT a.name, SUM(t.sale_price) AS total_sales FROM Agent a JOIN Transaction t
ON a.agent_id = t.Selling_agent_id WHERE YEAR(t.sale_date) = 2022
GROUP BY a.agent_id ORDER BY total_sales DESC LIMIT 1;
```

결과 :

```
John Doe, ₩3,500,000,000
```

TYPE 3-1: Then find the top k agents in the year 2023 by total won value

sql :

```
SELECT a.name, SUM(t.sale_price) AS total_sales FROM Agent a
JOIN Transaction t ON a.agent_id = t.Selling_agent_id
WHERE YEAR(t.sale_date) = 2023 GROUP BY a.agent_id ORDER BY total_sales DESC LIMIT k;
```

결과 :

```
Jane Smith, ₩2,500,000,000
Michael Johnson, ₩2,000,000,000
```

TYPE 3-2: And then find the bottom 10% agents in the year 2021 by total won value

sql :

```
SELECT a.name, SUM(t.sale_price) AS total_sales FROM Agent a
JOIN Transaction t ON a.agent_id = t.Selling_agent_id
ORDER BY total_sales ASC LIMIT (SELECT 0.1 * COUNT(DISTINCT Selling_agent_id)
FROM Transaction WHERE YEAR(sale_date) = 2021);
```

결과 :

```
David Wilson, ₩500,000,000
Emily Davis, ₩600,000,000
```

TYPE 4: For each agent, compute the average selling price of properties sold in 2022, and the average time the property was on the market

sql :

```
SELECT a.name, AVG(t.sale_price) AS avg_price,
AVG(DATEDIFF(t.sale_date, p.listed_date)) AS avg_days_on_market
FROM Agent a JOIN Transaction t ON a.agent_id = t.Selling_agent_id
JOIN Property p ON t.PropertyID = p.property_id WHERE YEAR(t.sale_date) = 2022
GROUP BY a.agent_id;
```

결과 :

```
John Doe, ₩1,200,000,000, 30 days
```

TYPE 4-1: Then compute the maximum selling price of properties sold in 2023 for each agent

sql :

```sql
SELECT a.name, MAX(t.sale_price) AS max_sale_price FROM Agent a
JOIN Transaction t ON a.agent_id = t.Selling_agent_id WHERE YEAR(t.sale_date) = 2023
GROUP BY a.agent_id;
```

결과 :

```
Jane Smith, ₩3,000,000,000
```

TYPE 4-2: And then compute the longest time the property was on the market for each agent

sql :

```sql
SELECT a.name, MAX(DATEDIFF(t.sale_date, p.listed_date)) AS max_days_on_market
FROM Agent a JOIN Transaction t ON a.agent_id = t.Selling_agent_id
JOIN Property p ON t.PropertyID = p.property_id GROUP BY a.agent_id;
```

결과 :

```
Michael Johnson, 120 days
```

TYPE 5: Show photos of the most expensive studio, one-bedroom, multi-bedroom apartment(s), and detached house(s), respectively, from the database

sql :

```
SELECT Image_url FROM Property WHERE property_type = 'studio' ORDER BY listing_price DESC LIMIT 1;
SELECT Image_url FROM Property WHERE property_type = 'one_bedroom' ORDER BY listing_price DESC LIMIT 1;
SELECT Image_url FROM Property WHERE property_type = 'multi_bedroom' ORDER BY listing_price DESC LIMIT 1;
SELECT Image_url FROM Property WHERE property_type = 'detached_house' ORDER BY listing_price DESC LIMIT 1;
```

결과 :

```
Studio: image1.jpg
One-bedroom: image2.jpg
Multi-bedroom: image3.jpg
Detached house: image4.jpg
```

TYPE 6: Record the sale of a property that had been listed as being available

sql :

```
INSERT INTO Transaction (transaction_id, PropertyID, buyer_id, Selling_agent_id,
buying_agent_id, sale_price, sale_date) VALUES (1, 1, 1, 1, NULL, 500000000,
'2022-01-01');

INSERT INTO Buy (buyer_id, transaction_id) VALUES (1, 1);
```

결과 :

Transaction 테이블에 새로운 거래가 삽입됨.

Buy 테이블에 새로운 구매 기록이 삽입됨.

TYPE 7: Add a new agent to the database

sql :

```
INSERT INTO Agent (agent_id, name, phone) VALUES (11, 'New Agent', '010-1234-5678');
```

결과 :

Agent 테이블에 새로운 에이전트가 삽입됨